```
          setComponent(await streamComponent(value, []));
        }}
      >
        <TextArea value={input} onChange={handleInputChange} />
      </form>
      <div>{component}</div>.
    </div>
  );
}
```

When the form is submitted, we call the streamComponent() server action.

The component state is rendered directly as components, allowing for dynamic updates as the streamable UI changes.

This `PageComponent` demonstrates how to integrate a streamable UI component generated by a server action into a React client component. It maintains two state variables: a component for the streamable UI and input for the user's text input. When the form is submitted, it calls the `streamComponent()` server action. This action generates a streamable UI component based on the input.

The result of the server action is set as the new value of the component state, and it is rendered in the DOM, allowing it to update as new content is streamed from the server. This approach enables real-time updates to the UI based on AI-generated content. The streamable UI component can update incrementally as new content is generated, without requiring full page reloads or explicit polling mechanisms.

Now what about when you want to stream real-time updates to a component from a server action while the content is being updated in real time? In such a case, we use the `createStreamableUI` function, which I will explain next.

4.1.6 Streaming React components with createStreamableUI

When building applications that require real-time updates to components based on server-side actions, the Vercel AI SDK provides the `createStreamableUI` function. This is particularly useful when content needs to be updated as it is processed.

The `createStreamableUI` function complements `streamUI` by providing a way to create and update UI components incrementally. This means you can update the UI components as new updates are received from the AI model, allowing for real-time updates based on ongoing AI processing. This improves perceived performance by showing partial results quickly and enhances the user experience.

For example, in the chatbot interface we are building, the AI generates responses incrementally. Each time a new response chunk is received, the server streams the updates as `streamable` React components that provide immediate feedback to the user without introducing any delays. Figure 4.7 shows how it works.

Compared to the `streamUI` function, the `createStreamableUI` does not really call the LLM provider with the prompt. It instead returns a value object that can be used in place of a React component. This object exposes several methods that the client can use to provide incremental updates:

- `update`—Replaces the existing content with the new content provided
- `done`—Signals that the streaming is complete